



Oficina de Git & Linux

UFERSA — Campus Pau dos Ferros

ECOP · 17 e 18 de Maio de 2026 · 16h–18h · Laboratório 2 - LTI

Para quem é esse material?

Essas notas acompanham os dois dias de oficina e servem como referência para consulta depois. Não precisa decorar nada — o objetivo é que você saia sabendo *onde* encontrar o que precisa e com prática suficiente pra não ter medo do terminal.

Sumário

- [Dia 2 — Git & GitHub](#)
 - [Por que controle de versão?](#)
 - [O ciclo Git](#)
 - [Repositório local — na prática](#)
 - [Ignorando arquivos — .gitignore](#)

- [Inspeccionando o histórico](#)
- [Desfazendo coisas](#)
- [Branches](#)
- [GitHub — indo para a nuvem](#)
- [Exercício final — Colaboração simulada](#)
- [Referência rápida](#)
- [Próximos passos](#)

Dia 2 — Git & GitHub

Ministrante: Gustavo Fontes

Horário: 16h00 – 18h00

Pré-requisito: ter participado do Dia 1 (ou saber navegar pelo terminal) e, de preferência, ter conta no GitHub.

1. Por que controle de versão?

O cenário sem Git

Quem nunca fez isso?

```
projeto-final.zip
projeto-final-v2.zip
projeto-final-AGORA-VAI.zip
projeto-final-entrega.zip
projeto-final-entrega-REAL.zip
```

Isso é controle de versão manual — e não escala. Problemas:

- Difícil saber o que mudou entre versões
- Impossível colaborar sem sobrescrever o trabalho dos outros
- Qualquer erro pode ser irreversível

O que Git resolve

Problema	Solução com Git
"Estraguei tudo e não consigo voltar"	<code>git revert</code> para desfazer commits com segurança
"Não sei o que mudou desde ontem"	<code>git diff</code> e <code>git log</code>
"Não consigo trabalhar com mais pessoas"	Branches + merge + pull requests
"Onde está a versão que funcionava na demo?"	Tags e hash de commits

Git ≠ GitHub

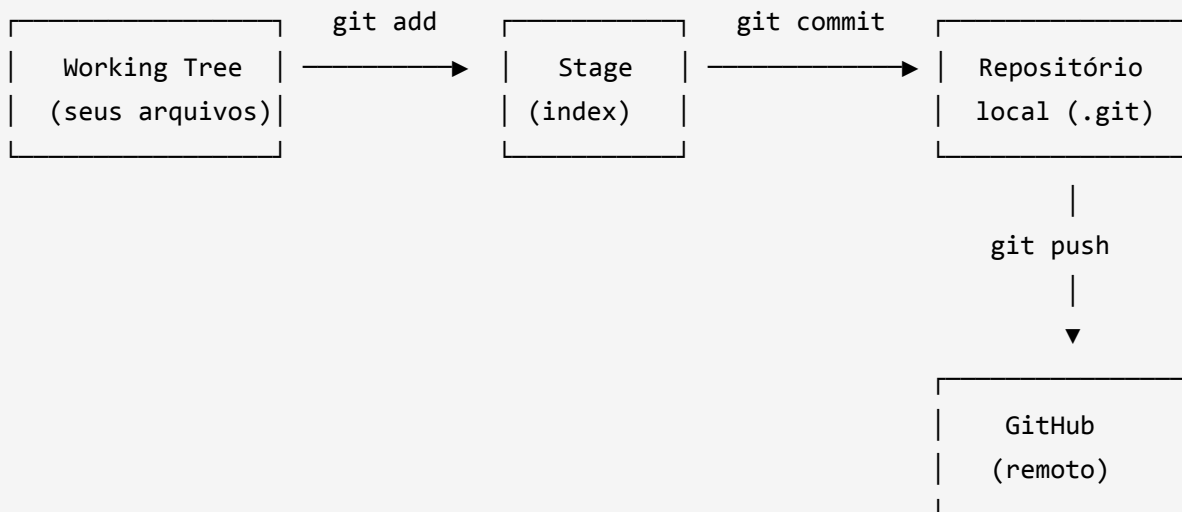
Git	GitHub
Ferramenta de controle de versão	Plataforma web para hospedar repositórios Git

Git	GitHub
Roda localmente no seu computador	Serviço na nuvem
Criado por Linus Torvalds (2005)	Fundado em 2008, comprado pela Microsoft em 2018
Open source	Serviço proprietário (com plano gratuito generoso)

Git é o motor. GitHub é uma das garagens onde você estaciona seu projeto. Existem alternativas: GitLab, Bitbucket, Codeberg.

2. O ciclo Git

Entender esse fluxo é a base de tudo:



- **Working Tree:** os arquivos que você vê e edita normalmente

- **Stage (index):** área de preparação — você escolhe o que vai para o próximo commit
 - **Repositório local:** o histórico de commits, armazenado na pasta `.git`
 - **Remoto (GitHub):** cópia do repositório na nuvem, acessível por qualquer pessoa autorizada
-

3. Repositório local — na prática

Configuração inicial (uma vez por máquina)

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
git config --global core.editor nano      # editor padrão (opcional)
git config --list                          # confirma as configurações
```

Iniciando um repositório

```
cd meu-projeto      # entrar na pasta do projeto do Dia 1
git init            # inicializa o repositório Git
ls -la              # agora existe uma pasta .git aqui
```

A pasta `.git` é onde o Git guarda todo o histórico. **Nunca delete essa pasta.**

O ciclo básico — passo a passo

1. Verificar o estado

```
git status
```

Saída típica:

```
On branch main

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    src/
    docs/

nothing added to commit but untracked files present
```

"Untracked" = Git sabe que o arquivo existe, mas não está rastreando ele ainda.

2. Adicionar ao stage

```
git add README.md      # adiciona um arquivo específico
git add src/           # adiciona uma pasta inteira
git add .              # adiciona tudo no diretório atual
```

Após o `git add`, o `git status` mostra:

Changes to be committed:

```
new file:   README.md
new file:   src/index.html
...
```

3. Criar o commit

```
git commit -m "feat: estrutura inicial do projeto"
```

Boas mensagens de commit:

```
git commit -m "feat: adiciona página inicial"      # nova funcionalidade
git commit -m "fix: corrige link quebrado no menu" # correção de bug
git commit -m "docs: atualiza README"              # documentação
git commit -m "refactor: reorganiza pastas src"    # refatoração
```

Fazendo mais commits — prática

```
# Editar o README
echo "## Tecnologias" >> README.md
echo "- HTML" >> README.md
echo "- CSS" >> README.md

# Ver o que mudou
git status
git diff README.md      # mostra as mudanças em detalhe

# Commitar
git add README.md
git commit -m "docs: adiciona seção de tecnologias ao README"

# Mais uma alteração
echo "<!-- Página inicial -->" > src/index.html
echo "<h1>Meu Projeto</h1>" >> src/index.html

git add src/index.html
git commit -m "feat: adiciona conteúdo básico ao index.html"
```

3a. Ignorando arquivos — `.gitignore`

Nem todo arquivo do projeto precisa ser versionado. Builds, dependências instaladas, arquivos com senhas — o Git deve ignorar esses.

O `.gitignore` é um arquivo de texto onde você lista padrões de arquivos que o Git deve ignorar completamente.

Criando um `.gitignore`

```
touch .gitignore  
nano .gitignore
```

Padrões comuns

```
# Dependências (Node.js)  
node_modules/  
  
# Builds  
dist/  
build/  
*.min.js  
  
# Variáveis de ambiente e senhas – NUNCA versione isso  
.env  
.env.local  
*.key  
  
# Arquivos do sistema operacional  
.DS_Store  
Thumbs.db  
  
# Arquivos do editor  
.vscode/  
.idea/  
*.swp
```

Testando se funciona

```
echo "minha-senha-secreta" > .env
git status                # .env aparece como untracked

echo ".env" >> .gitignore
git status                 # .env sumiu da lista!

git add .gitignore
git commit -m "chore: adiciona .gitignore"
```

Regra de ouro: nunca commite arquivos `.env` ou com credenciais. Se já commitou acidentalmente, a senha deve ser considerada comprometida e trocada imediatamente.

Templates prontos

Ao criar um repositório no GitHub, você pode escolher um template de `.gitignore` para sua linguagem — ele já vem configurado com os padrões mais comuns.

4. Inspeccionando o histórico

git log

```
git log           # histórico completo
git log --oneline  # uma linha por commit (mais legível)
git log --oneline --graph # com representação visual de branches
git log -5        # apenas os 5 commits mais recentes
```

Exemplo de saída do `git log --oneline`:

```
a3f2c1d (HEAD -> main) feat: adiciona conteúdo básico ao index.html
9b1e4f0 docs: adiciona seção de tecnologias ao README
c7a8d3e feat: estrutura inicial do projeto
```

- O hash (`a3f2c1d`) identifica o commit de forma única
- `HEAD` aponta para onde você está agora
- `main` é o nome da branch atual

git diff

```
git diff          # mudanças não staged (working tree vs stage)
git diff --staged  # mudanças staged (stage vs último commit)
git diff a3f2c1d 9b1e4f0 # diferença entre dois commits
```

git show

```
git show          # detalha o último commit
git show a3f2c1d  # detalha um commit específico
```

4a. Desfazendo coisas

Um dos maiores medos de iniciantes é "fazer algo errado e não conseguir voltar". Git foi feito exatamente para isso.

git restore — descarta mudanças não commitadas

```
git restore arquivo.txt  # descarta mudanças em um arquivo específico
git restore .            # descarta mudanças em todos os arquivos
git restore --staged arquivo # tira um arquivo do stage (sem perder as mudanças)
```

git revert — desfaz um commit de forma segura

`git revert` cria um *novo commit* que desfaz as mudanças de um commit anterior. O histórico fica preservado — ideal para projetos compartilhados.

```
git log --oneline
# a3f2c1d feat: adiciona seção de contato
# 9b1e4f0 docs: atualiza README

git revert a3f2c1d           # abre o editor para confirmar a mensagem
git revert a3f2c1d --no-edit # aceita a mensagem padrão automaticamente
```

git reset — move o HEAD (use com cuidado)

git reset é mais poderoso e desfaz o histórico. Deve ser usado **apenas em commits locais** que ainda não foram enviados ao GitHub.

```
git reset --soft HEAD~1    # desfaz o último commit, mantém arquivos no stage
git reset --mixed HEAD~1   # desfaz o último commit, tira do stage (padrão)
git reset --hard HEAD~1    # ⚠️ desfaz o commit E descarta todas as mudanças
```

Opção	Commit desfeito	Stage	Arquivos
--soft	✓	mantido	mantido
--mixed	✓	limpo	mantido
--hard	✓	limpo	descartado

Regra prática: git revert para branches compartilhadas (o que foi ao GitHub). git reset apenas para commits locais que ainda não saíram da sua máquina.

5. Branches

O que são branches?

Uma branch é uma linha paralela de desenvolvimento. Você pode trabalhar em uma nova funcionalidade sem afetar o código principal (`main`).

```
main:    A — B — C ————— F (merge)
              \           /
feature:      D — E ———
```

- Commits A, B, C aconteceram na `main`
- Você criou a branch `feature` a partir de C
- Fez commits D e E na feature
- Quando terminou, fez merge de volta na `main` (commit F)

Comandos de branch

```
git branch           # lista todas as branches
git branch nova-feature # cria uma nova branch
git switch nova-feature # muda para a branch (comando moderno)
git switch -c outra-feature # cria E já muda (comando moderno)
git branch -d nome     # deleta a branch (após merge)
```

Nota: `git checkout` ainda funciona para trocar de branch, mas desde o Git 2.23 (2019) o `git switch` é o comando preferido — ele é mais claro e faz só uma coisa.

```
# equivalentes antigos (ainda funcionam)
```

```
git checkout nova-feature      # igual a git switch
```

```
git checkout -b outra-feature  # igual a git switch -c
```

Prática com branches

Criar uma branch para desenvolver uma nova seção

```
git switch -c feature/contato
```

Fazer alterações nessa branch

```
echo "<h2>Contato</h2>" >> src/index.html
```

```
echo "<p>email@exemplo.com</p>" >> src/index.html
```

Commitar na branch

```
git add src/index.html
```

```
git commit -m "feat: adiciona seção de contato"
```

Ver a diferença entre as branches

```
git log --oneline --graph --all
```

Voltar para a main

```
git switch main
```

Verificar: a alteração NÃO está na main ainda

```
cat src/index.html
```

Fazer o merge

```
git merge feature/contato
```

Agora está!

```
cat src/index.html
```

Deletar a branch (já foi integrada)

```
git branch -d feature/contato
```

O que fazer quando tem conflito?

Um conflito acontece quando dois commits modificaram a mesma linha de formas diferentes. O Git marca o arquivo assim:

```
<<<<<< HEAD
conteúdo da branch atual
=====
conteúdo da branch que está sendo mergeada
>>>>>> feature/contato
```

Como resolver:

1. Abrir o arquivo com conflito
2. Escolher qual versão manter (ou combinar as duas)
3. Remover as marcações (<<<<<< , ===== , >>>>>>)
4. Salvar, `git add` e `git commit`

Na maioria dos projetos pequenos, conflitos são raros. Quando acontecem, não entre em pânico — o Git está te dizendo exatamente onde o problema está.

6. GitHub — indo para a nuvem

Criando um repositório no GitHub

1. Acesse github.com e faça login
2. Clique em **New repository** (botão verde ou ícone +)
3. Dê um nome ao repositório (ex: `meu-projeto`)

4. Deixe como **Public**
5. **Não** marque "Add a README" (já temos um)
6. Clique em **Create repository**

O GitHub vai mostrar os comandos. Siga a segunda opção ("*push an existing repository*"):

Autenticação — como o GitHub te identifica

Desde agosto de 2021, o GitHub **não aceita mais senha** para operações via terminal. Há duas formas de autenticar:

Opção 1 — Personal Access Token (mais simples para começar)

1. GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)**
2. Clique em **Generate new token**, dê um nome e marque a permissão `repo`
3. Copie o token gerado (ele só aparece uma vez)
4. Use esse token *no lugar da senha* quando o terminal pedir:

```
git push origin main
# Username: seu-usuario-github
# Password: <cole o token aqui – NÃO sua senha normal>
```

Para não precisar digitar toda vez:

```
git config --global credential.helper store # salva em arquivo (persistente)
# ou
git config --global credential.helper cache # salva temporariamente (15 min)
```

Opção 2 — Chave SSH (recomendada para uso contínuo)

```
# 1. Gerar o par de chaves
ssh-keygen -t ed25519 -C "seu@email.com"
# pressione Enter nas perguntas para aceitar os padrões

# 2. Copiar a chave pública
cat ~/.ssh/id_ed25519.pub
# Copie toda a saída

# 3. Adicionar no GitHub: Settings → SSH and GPG keys → New SSH key
# Cole o conteúdo copiado acima
```

Com SSH configurada, use a URL SSH ao adicionar o remote:

```
git remote add origin git@github.com:seu-usuario/meu-projeto.git
```

Para a oficina: use o Personal Access Token — é mais rápido de configurar agora. Para o dia a dia, vale configurar SSH uma vez e nunca mais precisar digitar credenciais.

Conectando o repositório local ao GitHub

```
# Adicionar o repositório remoto
git remote add origin https://github.com/seu-usuario/meu-projeto.git

# Verificar se foi configurado
git remote -v

# Enviar os commits para o GitHub
git push -u origin main
```

- `origin` é o nome que damos ao remoto (convenção, pode ser qualquer nome)
- `-u` configura o upstream — depois, só precisar de `git push`

Comandos do dia a dia com remoto

<code>git push</code>	# envia commits locais para o GitHub
<code>git pull</code>	# baixa e aplica mudanças do GitHub
<code>git fetch</code>	# baixa mudanças mas não aplica
<code>git clone URL</code>	# clona um repositório do zero
<code>git remote -v</code>	# lista os remotos configurados

Clonando um repositório

```
git clone https://github.com/usuario/repositorio.git
cd repositorio
```

Isso cria uma cópia local completa, já com todo o histórico.

Pull Requests (PR)

Um **Pull Request** é uma solicitação para que suas alterações (de uma branch ou fork) sejam incorporadas ao repositório principal. É o fluxo de colaboração padrão no GitHub.

Fluxo típico em um projeto colaborativo:

1. fork ou clone do repositório
2. criar branch: `git checkout -b feature/minha-funcionalidade`
3. fazer alterações e commits
4. `git push origin feature/minha-funcionalidade`
5. abrir Pull Request no GitHub
6. alguém revisa o código
7. PR é aprovado e mergeado na main

7. Exercício final — Colaboração simulada

O objetivo é simular o fluxo real de contribuição em um projeto open source.

O repositório da oficina foi criado pelo ministrante e está disponível no GitHub. Cada participante vai:

Passo 1 — Clonar o repositório da oficina

```
git clone https://github.com/adrianviniciuscs/curso-linuxgit-ecop2026.git
cd curso-linuxgit-ecop2026
ls
```

Passo 2 — Criar sua branch

```
git checkout -b participante/seu-nome
```

Use seu nome sem espaços e com letras minúsculas, por exemplo:

```
participante/joao-silva
```

Passo 3 — Fazer sua contribuição

Crie um arquivo com o seu nome `SEUNOME.txt` na pasta participantes e escreva o que achou da oficina:

```
cd participantes
echo "Seu Nome – o que achou da oficina" >> SEUNOME.txt
cat SEUNOME.txt      # verificar
```

Passo 4 — Commitar e enviar

```
git add SEUNOME.txt
git commit -m "feat: adiciona participante Seu Nome"
git push origin participante/seu-nome
```

Passo 5 (bônus) — Abrir um Pull Request

1. Acesse o repositório no GitHub
2. Você vai ver um banner: *"Compare & pull request"*
3. Clique, adicione uma descrição e abra o PR
4. O ministrante vai fazer o merge ao vivo!

Referência rápida

Linux

Comando	O que faz
<code>pwd</code>	Mostra o diretório atual
<code>ls</code>	Lista arquivos e pastas
<code>ls -la</code>	Lista com detalhes e ocultos
<code>cd pasta/</code>	Entra em um diretório
<code>cd ..</code>	Volta um nível
<code>cd ~</code>	Vai para o home
<code>mkdir nome</code>	Cria um diretório
<code>mkdir -p a/b/c</code>	Cria estrutura aninhada
<code>touch arq</code>	Cria arquivo vazio
<code>echo "txt" > arq</code>	Escreve em arquivo
<code>cat arq</code>	Exibe conteúdo do arquivo
<code>cp origem dest</code>	Copia arquivo

Comando	O que faz
<code>cp -r dir/ dest/</code>	Copia diretório
<code>mv origem dest</code>	Move ou renomeia
<code>rm arq</code>	Remove arquivo
<code>rm -r pasta/</code>	Remove pasta e conteúdo
<code>man cmd</code>	Abre o manual do comando
<code>history</code>	Histórico de comandos
<code>clear</code>	Limpa a tela

Git

Comando	O que faz
<code>git config --global user.name "Nome"</code>	Define seu nome
<code>git config --global user.email "e@mail"</code>	Define seu e-mail
<code>git init</code>	Inicializa um repositório
<code>git status</code>	Mostra o estado dos arquivos
<code>git add arquivo</code>	Adiciona arquivo ao stage
<code>git add .</code>	Adiciona tudo ao stage
<code>git commit -m "msg"</code>	Cria um commit

Comando	O que faz
<code>git log</code>	Exibe histórico de commits
<code>git log --oneline</code>	Histórico resumido
<code>git diff</code>	Mostra mudanças não staged
<code>git diff --staged</code>	Mostra mudanças staged
<code>git show</code>	Detalha o último commit
<code>git restore arquivo</code>	Descarta mudanças não commitadas
<code>git restore --staged arquivo</code>	Tira arquivo do stage
<code>git revert hash</code>	Desfaz commit (seguro, preserva histórico)
<code>git reset --soft HEAD~1</code>	Desfaz commit, mantém stage
<code>git reset --hard HEAD~1</code>	Desfaz commit e descarta mudanças
<code>git branch</code>	Lista branches
<code>git branch nome</code>	Cria uma nova branch
<code>git switch nome</code>	Muda para a branch (moderno)
<code>git switch -c nome</code>	Cria e já muda de branch (moderno)
<code>git merge nome</code>	Faz merge na branch atual
<code>git branch -d nome</code>	Deleta uma branch
<code>git remote add origin URL</code>	Conecta ao repositório remoto

Comando	O que faz
<code>git remote -v</code>	Lista remotos configurados
<code>git push -u origin main</code>	Envia commits (primeira vez)
<code>git push</code>	Envia commits
<code>git pull</code>	Atualiza do remoto
<code>git clone URL</code>	Clona um repositório

Erros comuns e soluções

Erro	Causa provável	Solução
<code>not a git repository</code>	Pasta sem <code>.git</code>	Rode <code>git init</code> ou entre na pasta correta
<code>nothing to commit</code>	Nenhum arquivo no stage	Rode <code>git add .</code> antes do commit
<code>rejected (non-fast-forward)</code>	Remoto tem commits que você não tem	Rode <code>git pull</code> antes do <code>git push</code>
<code>detached HEAD</code>	Você fez checkout num commit direto	Rode <code>git checkout main</code>
<code>CONFLICT</code> durante merge	Dois commits editaram a mesma linha	Abra os arquivos, resolva e commite

Próximos passos

Você saiu da oficina sabendo usar o terminal e versionar projetos com Git. O que vem depois?

Recursos gratuitos

Recurso	Link	Para quê
Pro Git (livro)	git-scm.com/book/pt-br	Referência completa em português
Learn Git Branching	learngitbranching.js.org	Visualizar branches interativamente
Oh Shit, Git!	ohshitgit.com/pt_BR	Quando der errado
GitHub Student Pack	education.github.com/pack	Ferramentas gratuitas para estudantes
The Linux Command Line	linuxcommand.org	Aprofundar no terminal

Próximo nível em Git

Quando o básico estiver confortável, explore:

- `git stash` — guardar alterações temporariamente sem commitar
- `git rebase` — reescrever histórico de forma mais limpa (use com cuidado)
- `git cherry-pick` — aplicar um commit específico em outra branch
- GitHub Actions — automatizar testes e deploys (CI/CD)
- Conventional Commits — padrão de mensagens de commit (`feat:` , `fix:` , `chore:` ...)

Próximo nível em Linux

- `grep` — buscar texto em arquivos
- `|` (pipe) — encadear comandos
- `chmod` — permissões de arquivos
- `nano` ou `vim` — editar arquivos pelo terminal
- `ssh` — conectar em servidores remotos
- `bash scripting` — automatizar tarefas com scripts

Uma dica final: a melhor forma de consolidar o que aprendeu é ter um projeto real para versionar. Pode ser qualquer coisa — um site simples, um script, notas de aula. O que importa é usar Git todo dia até virar hábito.

Oficina de Git & Linux · UFERSA — Campus Pau dos Ferros

Material produzido para uso livre pelos participantes